

FORTRAN 90

Estructuras de control

Variables Lógicas

Una variable **LOGICA** sólo puede guardar dos valores **.TRUE.** o **.FALSE.**, y no puede guardar ningún otro tipo de variable. Se usa **T** or **F** para una variable de tipo **LOGICAL** que es leído por **READ(*,*)**. **WRITE(*,*)** imprime **T** o **F** para un valor **.TRUE.** o **.FALSE.**, respectivamente.

Ejemplo:

```
LOGICAL, PARAMETER :: Test = .TRUE.
LOGICAL :: C1, C2
C1 = .true. ! correct
C2 = 123    ! Wrong
READ(*,*) C1, C2
C2 = .false.
WRITE(*,*) C1, C2
```

Operadores Relacionales

Fortran 90 tiene seis operadores relacionales: **<**, **<=**, **>**, **>=**, **==**, **/=**. Cada una de estas operaciones toma dos expresiones, compara sus valores y obtiene uno de los dos resultados posibles **.TRUE.** o **.FALSE.**

Así, **a < b < c** está mal ya que, **a < b** da un resultado de tipo **LOGICAL** y **c** es un tipo **REAL** o un tipo **INTEGER**. En el tipo **COMPLEX** solo pueden usarse **==** y **/=**. Dos valores de tipo **LOGICAL** puede usarse en la operación lógica **.EQV.** o **.NEQV.** para una comparación de igualdad o desigualdad.

Los operadores relacionales tienen la **menor** precedencia o prioridad que los operadores aritméticos y del operador **//**. Así, **3 + 5 > 10** es **.FALSE.** y **"a" // "b" == "ab"** es **.TRUE.**

Para el caso de "strings" y su comparación relacional con el objetivo de tener la **máxima portabilidad**, solo se consideran los siguientes órdenes para las letras y los dígitos (ojo: vistos como strings).

Así, **"A" < "X"**, **'f' <= "u"**, y **"2" < "7"** son **.TRUE.** Pero, no sabemos los resultados de **"S" < "s"** y **"t" >= "%"**. Sin embargo, la igualdad o desigualdad tal como **"S" /= "s"** y **"t" == "5"** son correctos. Entonces se cumple lo siguiente:

```
A < B < C < D < E < F < G < H < I < J < K < L < M < N
    < O < P < Q < R < S < T < U < V < W < X < Y < Z
```

```
a < b < c < d < e < f < g < h < i < j < k < l < m < n
```

`< o < p < q < r < s < t < u < v < w < x < y < z`
`0 < 1 < 2 < 3 < 4 < 5 < 6 < 7 < 8 < 9`

Reglas de comparación:

Se comienza barriendo del primer carácter del string. Si son iguales se pasa la comparación al siguiente. Si no hay otro carácter la comparación de los string da igual (e.g., `"abc" == "abc"`)

Si uno de los strings no tiene más caracteres para comparar, el de longitud más corta es el más pequeño (e.g., `"ab" < "abc"` es `.TRUE.`)

Si en dos strings no son iguales el string que tiene el character mas pequeño es el más pequeño (e.g., `"abcd"` es mas pequeño que `"abct"`).

Operadores lógicos

Tenemos 5 operadores lógicos en Fortran 90:

`.NOT.`, `.OR.`, `.AND.`, `.EQV.` y `.NEQV.`

Ahora `.NOT.` es le de más alta prioridad seguido por, `.OR.` luego `.AND.`, `.EQV.` y `.NEQV.` son los de más baja prioridad.

Hay que recalcar que el operador `.NOT.` se evalúa de *derecha a izquierda*.

Si ambos operadores del operdor `.EQV.` (*equivalencia*) son el mismo, `.EQV.` dará como resultado `.TRUE...` `.NEQV.` es lo opuesto a `.EQV.` (*no equivalente*).

Si los operandos de `.NEQV.` tiene diferentes valores, `.NEQV.` resulta `.TRUE.`

Si las variables tipo `INTEGER` `m`, `n`, `x` y `y` tienen los valores 3, 5, 4 y 2, respectivamente.

Ejemplos:

```

.NOT. (m > n .AND. x < y) .NEQV. (m <= n .AND. x >= y)
.NOT. (3 > 5 .AND. 4 < 2) .NEQV. (3 <= 5 .AND. 4 >= 2)
.NOT. (.FALSE. .AND. 4 < 2) .NEQV. (3 <= 5 .AND. 4 >= 2)
.NOT. (.FALSE. .AND. .FALSE.) .NEQV. (3 <= 5 .AND. 4 >= 2)
.NOT. .FALSE. .NEQV. (3 <= 5 .AND. 4 >= 2) .NOT. es más
                    prioritario que .NEQV.
.TRUE. .NEQV. (3 <= 5 .AND. 4 >= 2)
.TRUE. .NEQV. (.TRUE. .AND. 4 >= 2)
.TRUE. .NEQV. (.TRUE. .AND. .TRUE.)
.TRUE. .NEQV. .TRUE.
.FALSE.

```

Instrucción IF-THEN-ELSE

La instrucción if-then-else en Fortran 90 tiene tres formas. Pero la más completa de estas y de la que se pueden desprender las otras formas es **IF-THEN-ELSEIF-END IF**.

La instrucción **IF-THEN-ELSE-IF-END IF** funciona como sigue:

Las expresiones lógicas son evaluadas secuencialmente (*i.e.*, de arriba abajo).

La instrucción que corresponda a la expresión evaluada como **.TRUE.** será ejecutada. De otra manera se sigue a la siguiente instrucción **ELSE** y se evalúa la siguiente expresión lógica. Si ninguna es evaluada como verdadera se ejecutará el último la instrucción correspondiente al último **ELSE**:

```
IF (logical-expression-1) THEN
statement sequence 1
ELSE IF (logical-expression-2) THEN
statement sequence 2
ELSE IF (logical-expression-3) THEN
statement sequence 3
ELSE IF (.....) THEN
.....
ELSE
statement sequence ELSE
END IF
```

Dos ejemplos:

Encuentra el mínimo de a, b y c
Y guarda el resultado en Result

```
IF (a < b .AND. a < c) THEN
Result = a
ELSE IF (b < a .AND. b < c) THEN
Result = b
ELSE
Result = c
END IF
```

El clásico ejemplo de las calificaciones con letra

```
INTEGER :: x
CHARACTER(LEN=1) :: Grade
IF (x < 50) THEN
Grade = 'F'
ELSE IF (x < 60) THEN
Grade = 'D'
ELSE IF (x < 70) THEN
Grade = 'C'
ELSE IF (x < 80) THEN
Grade = 'B'
ELSE
Grade = 'A'
END IF
```

En las instrucciones anteriores el **ELSE-IF** y **ELSE** resulta que pueden ser opcionales. Si el **ELSE** falta en el conjunto de instrucciones y ninguna de la expresiones lógicas tienen **.TRUE.**, como su evaluación, entonces **IF-THEN-ELSE** no tiene ningún efecto (ninguna instrucción es evaluada).

No hay **ELSE-IF**:

```
IF (logical-expression-1) THEN
statement sequence 1
ELSE
statement sequence ELSE
END IF
```

No hay **ELSE**:

```
IF (logical-expression-1) THEN
statement sequence 1
ELSE IF (logical-expression-2) THEN
statement sequence 2
ELSE IF (logical-expression-3) THEN
statement sequence 3
ELSE IF (.....) THEN
.....
END IF
```

La forma más sencilla del “IF” es:

Si la expresión lógica es **.TRUE.**, es la instrucción a su lado es ejecutada. De otra manera, la ejecución continúa. La instrucción puede ser de inclusive de input/output.

IF (logical-expression) *statement*

Ejemplo:

```
Smallest = b
IF (a < b) Smallest = a
```

Otro ejemplo:

```
Cnt = Cnt + 1
IF (MOD(Cnt,10) == 0) WRITE(*,*) Cnt
```

La instrucción SELECT CASE

Fortran 90 tiene la instrucción **SELECT CASE** que se utiliza para la ejecución selectiva si los criterios de selección son basadas en valores simples en tipos **INTEGER**, **LOGICAL** o **CHARACTER**. Y no es aplicado al tipo **REAL**.

<pre>SELECT CASE (<i>selector</i>) CASE (<i>label-list-1</i>) <i>statements-1</i> CASE (<i>label-list-2</i>) <i>statements-2</i> CASE (<i>label-list-3</i>) <i>statements-3</i> other cases CASE (<i>label-list-n</i>) <i>statements-n</i> CASE DEFAULT <i>statements-DEFAULT</i> END SELECT</pre>	<i>El selector es una expresión como INTEGER, LOGICAL o CHARACTER</i> <i>label-list es un conjunto de constantes o PARAMETERS del tipo del selector</i> <i>los statements son instrucciones ejecutables una o más</i>
--	--

Las “*label-list*” son una lista que pueden tener las siguientes formas:

value → un valor específico

value1 : value2 → valores entre **value1** y **value2**, incluyendo al **value1** y al **value2** y **value1 <= value2**

value1 : → valores más grandes o iguales a **value1**

: value2 → valores menores o iguales al **value2**

Recordar: los **value**, **value1** **value2** deberán ser constantes o **PARAMETERS**.

El **SELECT CASE** funciona como sigue:
 Compara el valor de **selector** con las etiquetas en cada caso. Si alguna coincide se ejecutan las correspondientes **instrucciones o statements**.
 Si no se encuentra alguna coincidencia se lleva al **CASE DEFAULT** y entonces se ejecutan, los **statements- por DEFAULT**. Ejecuta la siguiente instrucción que sigue **SELECT CASE**.

```
SELECT CASE (selector)
CASE (label-list-1)
statements-1
CASE (label-list-2)
statements-2
CASE (label-list-3)
statements-3
..... other cases .....
CASE (label-list-n)
statements-n
CASE DEFAULT
statements-DEFAULT
END SELECT
```

Es opcional

Algunas notas importantes :

Los valores en **label-lists** deberán ser únicas. De otra manera no se sabrá, en cual **CASE** fue seleccionado. El **CASE DEFAULT** será usado siempre que sea posible, debido a que garantiza que haya un lugar para hacer algo (e.g., un mensaje de error) si no se encuentra alguna coincidencia. El **CASE DEFAULT** puede estar en cualquier parte del bloque del **SELECT CASE**; pero, es preferible colocarlo al final de la lista del **CASE**.

Dos ejemplos de la instrucción **CASE** :

```
CHARACTER(LEN=4) :: Title
INTEGER :: DrMD = 0, PhD = 0
INTEGER :: MS = 0, BS = 0
INTEGER :: Others = 0
SELECT CASE (Title)
CASE ("DrMD")
  DrMD = DrMD + 1
CASE ("PhD")
  PhD = PhD + 1
CASE ("MS")
  MS = MS + 1
CASE ("BS")
  BS = BS + 1
CASE DEFAULT
  Others = Others + 1
END SELECT
```

El siguiente ejemplo:

```
CHARACTER(LEN=1) :: c
SELECT CASE (c)
CASE ('a' : 'j')
    WRITE(*,*) 'First ten letters'
CASE ('l' : 'p', 'u' : 'y')
    WRITE(*,*) &
    'One of l,m,n,o,p,u,v,w,x,y'
CASE ('z', 'q' : 't')
    WRITE(*,*) 'One of z,q,r,s,t'
CASE DEFAULT
    WRITE(*,*) 'Other characters'
END SELECT
```

He aquí un ejemplo más complejo:

```
INTEGER :: Number, Range
SELECT CASE (Number)
CASE ( : -10, 10 : )
    Range = 1
CASE (-5:-3, 6:9)
    Range = 2
CASE (-2:2)
    Range = 3
CASE (3, 5)
    Range = 4
CASE (4)
    Range = 5
CASE DEFAULT
    Range = 6
END SELECT
```

El último ejemplo completo:

```
PROGRAM CharacterTesting
IMPLICIT NONE
CHARACTER(LEN=1) :: Input
READ(*,*) Input
SELECT CASE (Input)
CASE ('A' : 'Z', 'a' : 'z') ! rule out letters
WRITE(*,*) 'A letter is found : "', Input, '"'
SELECT CASE (Input) ! a vowel ?
CASE ('A', 'E', 'I', 'O', 'U', 'a', 'e', 'i', 'o', 'u')
WRITE(*,*) 'It is a vowel'
CASE DEFAULT ! it must be a consonant
WRITE(*,*) 'It is a consonant'
END SELECT
CASE ('0' : '9') ! a digit
WRITE(*,*) 'A digit is found : "', Input, '"'
CASE ('+', '-', '*', '/') ! an operator
WRITE(*,*) 'An operator is found : "', Input, '"'
CASE (' ') ! space
WRITE(*,*) 'A space is found : "', Input, '"'
CASE DEFAULT ! something else
WRITE(*,*) 'Something else found : "', Input, '"'
END SELECT
END PROGRAM CharacterTesting
```

Este programa lee un carácter y determina si es una vocal, una consonante, un dígito, uno de los cuatro operadores aritméticos, un espacio o algo más (i.e., %, \$, @, etc).

Estructura DO "The Counting"

Fortran 90 tiene dos formas de estructuras DO: el "counting DO" y el DO general. El "counting DO" tiene la siguiente forma:

```
DO control-var = inicial, final [, step]
```

statements

```
END DO
```

La variable `control-var` es una variable `INTEGER`, `inicial`, `final` y `step` son expresiones de tipo `INTEGER`; sin embargo, `step` *no puede ser cero*. Si `step` es omitido, su valor por default es 1. "*Statements*" son instrucciones ejecutables.

Si `step` es positivo: La variable `control-var` recibe el valor de `initial`. Si el valor de `control-var` es menor o igual que el valor de `final`, los "*statements*" son ejecutados. Entonces, el valor de `step` es sumado a `control-var`, y de nuevo se compara el valor de la variable `control-var` y `final`. Si el valor de `control-var` es más grande que el valor `final`, el "DO-loop" se completa y luego se ejecuta lo que sigue al `END DO`.

Si `step` es negativo: La variable `control-var` recibe el valor de `initial`. Si el valor de `control-var` es más grande o igual que el valor de `final`, las instrucciones (*statements*) son ejecutadas. Entonces el valor `step` es sumada (restada) a `control-var`, y de nuevo se somparan los valores de `control-var` y `final`. Si el valor de `control-var` es menor que el valor de `final`, el DO-loop se completa y la instrucción que sigue al `END DO` es ejecutada.

Dos ejemplos sencillos:

"Números pares entre el 1 & N"

```
INTEGER :: N, k
READ(*,*) N
WRITE(*,*) "Odd number between 1 and ", N
DO k = 1, N, 2
WRITE(*,*) k
END DO
```

“Factorial de N”

```
INTEGER, PARAMETER :: LONG = SELECTED_INT_KIND(15)
INTEGER(KIND=LONG) :: Factorial, i, N
READ(*,*) N
Factorial = 1_LONG
DO i = 1, N
  Factorial = Factorial * i
END DO
WRITE(*,*) N, "! = ", Factorial
```

Notas importantes:

El tamaño de paso **step** *no puede ser cero*. Nunca se debe cambiar el valor de las variables **control-var** y **initial**, **final**, y **step**.

Para un conteo en reversa en un DO-loop, el **step** deberá ser negativo. Así tenemos que si, “do i = 10, -10” no es un DO-loop decremental, y las instrucciones “*statements*” no serán ejecutadas en ningún momento. En Fortran 77 se permitía el tipo **REAL** en las variables en el DO; pero no se deben usar porque no es seguro.

Estructura DO general

El DO-loop general tiene la siguiente forma:

```
DO
  Statements
END DO
```

Los “*statements*” o instrucciones con esta forma simplemente serán ejecutadas repetidamente. Para terminar el DO-loop, se hace el uso de las instrucciones **EXIT** o **CYCLE**.

La instrucción **EXIT** tendrá el control del flujo (i.e., la salida) del **END DO**.

La instrucción **CYCLE** empieza la siguiente iteración (i.e., ejecutando los *statements* de nuevo).

```
REAL, PARAMETER :: Lower = -1.0, Upper = 1.0, Step = 0.25
REAL :: x
x = Lower           ! inicializa la variable de control
DO
  IF (x > Upper) EXIT ! es el valor final ?
  WRITE(*,*) x
  x = x + Step       ! se incrementa el paso
END DO
```

Otro ejemplo:

```
INTEGER :: Input
DO
WRITE(*,*) 'Type in an integer in [0, 10] please --> '
READ(*,*) Input
IF (0 <= Input .AND. Input <= 10) EXIT
WRITE(*,*) 'Your input is out of range. Try again'
END DO
```

Ejemplo completo

La función exponencial tiene una serie infinita de la forma:

$$\exp(x) = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots + \frac{x^i}{i!} + \dots$$

Se suma cada término hasta que el valor absoluto es menor que una tolerancia de 0.00001.

Nota:
$$\frac{x^{i+1}}{(i+1)!} = \left(\frac{x^i}{i!} \right) * \left(\frac{x}{i+1} \right)$$

```
PROGRAM Exponential
IMPLICIT NONE
INTEGER :: Count          ! # de terminos usados
REAL :: Term              ! un termino
REAL :: Sum               ! la suma
REAL :: X                 ! el input x
REAL, PARAMETER :: Tolerance = 0.00001 ! tolerance
READ(*,*) X              ! lee en x
Count = 1                 ! el primer termino es 1
Sum = 1.0                 ! asi, la suma empieza con 1
Term = X                  ! el Segundo termino es x
DO                        ! para cada termino
  IF (ABS(Term) < Tolerance) EXIT ! si es muy pequeño,exit
  Sum = Sum + Term             ! de otra manera, aumenta a sum
  Count = Count + 1 ! cuenta que indica el siguiente termino
  Term = Term * (X / Count) ! calcula el valor del siguiente
END DO
WRITE(*,*) 'After ', Count, ' iterations:'
WRITE(*,*) ' Exp(', X, ') = ', Sum
WRITE(*,*) ' From EXP() = ', EXP(X)
WRITE(*,*) ' Abs(Error) = ', ABS(Sum - EXP(X))
END PROGRAM Exponential
```

Otro ejemplo “detector de primos”:

Si tenemos un entero $n \geq 2$ será **primo** si el único divisor de este entero son 1 y si mismo.

Si $n = 2$, este es un primo.

Si $n > 2$ es par (i.e., $\text{MOD}(n,2) == 0$), no es un primo.

Si n es impar, then:

Si los números impares entre 3 y $n-1$ no pueden dividir a n , n es un primo!

¿Pero es necesario ir hasta $n-1$ probando la divisibilidad? No, $\text{SQRT}(n)$ es suficiente.

```
INTEGER :: Number      ! el numero entrante
INTEGER :: Divisor      ! el divisor corriendo
READ(*,*) Number       ! se lee el input
IF (Number < 2) THEN    ! no es primo Si < 2
  WRITE(*,*) 'Illegal input'
ELSE IF (Number == 2) THEN      ! si es = 2 es primo
  WRITE(*,*) Number, ' is a prime'
ELSE IF (MOD(Number,2) == 0) THEN ! no es un primo si es par
  WRITE(*,*) Number, ' is NOT a prime'
ELSE
  Divisor = 3      ! si es un numero impar
                  ! el divisor empieza con 3
DO                ! se divide el numero input
  IF (Divisor*Divisor > Number .OR. MOD(Number, Divisor) == 0) EXIT
  Divisor = Divisor + 2 ! se incrementa al proximo impar
END DO
IF (Divisor*Divisor > Number) THEN ! en que condicion cae?
  WRITE(*,*) Number, ' is a prime'
ELSE
  WRITE(*,*) Number, ' is NOT a prime'
END IF
END IF
```

Esto es mejor $\text{SQRT}(\text{REAL}(\text{Divisor})) > \text{Number}$

El programa anterior puede ser modificado para encontrar todos los números primos entre 2 n .

Manejando el “End-of-File”

Es muy frecuente que no conozcamos el número de datos que introducimos a un programa. Fortran usa la instrucción `IOSTAT=` para manejar un error de I/O :

`READ(*,*,IOSTAT=v) v1, v2, ..., vn`

En el ejemplo anterior, v es una variable de tipo `INTEGER`.

Después de la ejecución del `READ(*,*)`:

Si $v = 0$, `READ(*,*)` significa que este fué ejecutado exitosamente

Si $\nu > 0$, un error ocurrió durante la ejecución del `READ (*, *)` y no todas las variables recibieron valores.

Si $\nu < 0$, encontró un fin de archivo, y no todas la variables reciben valores.

<pre> INTEGER :: io, x, sum sum = 0 DO READ(*,*,IOSTAT=io) x IF (io > 0) THEN WRITE(*,*) 'Check input Something& was wrong' EXIT ELSE IF (io < 0) THEN WRITE(*,*) 'The total is ', sum EXIT ELSE sum = sum + x END IF END DO </pre>	Input 1 3 4 Input 1 & 4	Output The total is 8 Output no output
---	---	--

Como ejemplo vamos a calcular los promedios aritmético, geométrico ya armónico de un número desconocido de valores:

$$\text{Promedio aritmético} = \frac{x_1 + x_2 + \dots + x_n}{n}$$

$$\text{Promedio geométrico} = \sqrt[n]{x_1 \times x_2 \times \dots \times x_n}$$

$$\text{Promedio armónico} = \frac{n}{\frac{1}{x_1} + \frac{1}{x_2} + \dots + \frac{1}{x_n}}$$

Solo números **positivos** serán considerados.

```

PROGRAM ComputingMeans
IMPLICIT NONE
REAL      :: X
REAL      :: Sum, Product, InverseSum
REAL      :: Arithmetic, Geometric, Harmonic
INTEGER   :: Count, TotalValid
INTEGER   :: IO ! para el IOSTAT=

Sum        = 0.0
Product    = 1.0
InverseSum = 0.0
TotalValid = 0
Count      = 0

DO
  READ(*,*,IOSTAT=IO) X ! lee datao
  IF (IO < 0) EXIT      ! IO < 0 significa end-of-file alcanzado
    Count = Count + 1 ! de otra manera, se obtiene valor
  IF (IO > 0) THEN      ! IO > 0 significa que algo anda mal
    WRITE(*,*) 'ERROR: something wrong in your input'
    WRITE(*,*) 'Try again please'
  ELSE                  ! IO = 0 significa todo normal
    WRITE(*,*) 'Input item ', Count, ' --> ', X
    IF (X <= 0.0) THEN
      WRITE(*,*) 'Input <= 0. Ignored'
    ELSE
      TotalValid = TotalValid + 1
      Sum = Sum + X
      Product = Product * X
      InverseSum = InverseSum + 1.0/X
    END IF
  END IF
END DO
WRITE(*,*)
IF (TotalValid > 0) THEN
  Arithmetic = Sum / TotalValid
  Geometric = Product**(1.0/TotalValid)
  Harmonic = TotalValid / InverseSum
  WRITE(*,*) '# of items read --> ', Count
  WRITE(*,*) '# of valid items -> ', TotalValid
  WRITE(*,*) 'Arithmetic mean --> ', Arithmetic
  WRITE(*,*) 'Geometric mean --> ', Geometric
  WRITE(*,*) 'Harmonic mean --> ', Harmonic
ELSE
  WRITE(*,*) 'ERROR: none of the input is positive'
END IF
END PROGRAM ComputingMeans

```